

Кам'яний будинок

Арбен успадкував чудову ділянку землі на пагорбах, що височіють над Гірокастрою. Натхненний знаменитими будинками міста з кам'яними дахами, він планує побудувати власний традиційний будинок.

Земля подана як квадратна сітка $N \times N$, рядки та стовпці якої пронумеровані від 1 до N . Склеяста місцевість має особливість: у кожному рядку та кожному стовпці сітки міститься рівно один великий нерухомий валун.

Щоб закласти фундамент свого будинку, Арбен має вибрати **порожню квадратну підсітку** — область розміру $S \times S$, яка не містить жодного валуна.

Нехай M — максимальне ціле число таке, що для *будь-якого* розташування валунів, яке задовольняє наведені вище умови, гарантовано існує порожня квадратна підсітка розміру $M \times M$. Арбен хоче знайти *будь-яку* порожню квадратну підсітку розміру щонайменше $M \times M$.

Щоб обстежити землю, Арбен просить про допомогу свою подругу Бесу. Беса принесла спеціальний вимірювальний пристрій під назвою **Gurëmatës** ("лічильник каменів").

За одну операцію Беса може налаштувати Gurëmatës на сканування прямокутної підсітки, заданої її верхнім лівим кутом (x_1, y_1) та нижнім правим кутом (x_2, y_2) . Пристрій повертає точну кількість валунів у цій області. Gurëmatës живиться від маленької батареї, і кожне сканування споживає багато енергії, тому Беса має використовувати його якомога рідше.

Напишіть програму, яка за допомогою Gurëmatës знаходить порожню квадратну підсітку розміру $S \times S$ з $S \geq M$ та повідомляє її розташування.

Деталі реалізації

Ви маєте реалізувати таку процедуру:

```
std::vector < int > solve(int N);
```

- N : розмір сітки.
- Функція має повернути рівно чотири цілі числа $\{x_1, y_1, x_2, y_2\}$: верхній лівий та нижній правий кути знайденої порожньої квадратної підсітки. Має виконуватися

умова $x_2 - x_1 = y_2 - y_1 = S - 1$ з $S \geq M$.

- Ця функція викликається рівно один раз для кожного тестового випадку.

Функція може викликати таку функцію:

```
int ask(int x1, int y1, int x2, int y2);
```

- x_1, y_1 : рядок і стовпець верхнього лівого кута сканованої підсітки.
- x_2, y_2 : рядок і стовпець нижнього правого кута сканованої підсітки.
- Мають виконуватися умови $1 \leq x_1 \leq x_2 \leq N$ та $1 \leq y_1 \leq y_2 \leq N$. Інакше ваша програма негайно завершується й отримує 0 балів за цей тестовий випадок.
- Функція повертає кількість валунів у сканованій підсітці.

Грейдер не є адаптивним: позиції валунів фіксуються до виклику `solve` і не залежать від ваших викликів `ask`.

Обмеження

- $2 \leq N \leq 500$
- Ви можете зробити не більше $2.5 \cdot 10^5$ викликів `ask`.

Підзадачі

Підзадача	Бали	Додаткові обмеження
1	15	$N \leq 40$
2	25	$N \leq 150$
3	60	Без додаткових обмежень.

Оцінювання

Якщо `solve` повертає некоректну область, робить некоректний виклик `ask` або викликає `ask` більше ніж $2.5 \cdot 10^5$ разів, ви отримуєте 0 балів за цей тестовий випадок.

Інакше нехай Q — кількість викликів `ask`. За цей тестовий випадок ви отримуєте бали підзадачі, помножені на $\min(1, f(Q))$, де

$$f(Q) = \begin{cases} \max(0, -0.05 \cdot Q + 1.55), & \text{if } Q \leq 15 \\ \max(0, -0.19 \cdot \log_{10}(Q) + 1.02), & \text{if } Q > 15 \end{cases}$$

Ваш бал за підзадачу — це мінімальний бал серед усіх тестових випадків цієї підзадачі.
Підсумковий бал — це сума балів за підзадачі.

Приклад грейдера

Приклад грейдера читає вхідні дані в такому форматі:

```
N
c_1 c_2 ... c_N
```

Тут c_i — стовпець валуна в рядку i ($1 \leq c_i \leq N$).

Приклад грейдера виводить один рядок, що містить множник $\min(1, f(Q))$, отриманий вашими запитами, або 0, якщо повернена підсітка або будь-який виклик `ask` є некоректним.

Приклади

Приклад 1

Вхідні дані прикладу грейдера:

```
4
1 2 3 4
```

Валуни розташовані в $(1, 1)$, $(2, 2)$, $(3, 3)$ та $(4, 4)$. Ваша програма цього не знає.

Грейдер викликає `solve(4)`. Одна з можливих послідовностей викликів:

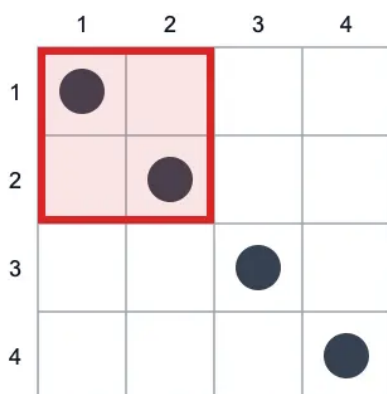
- `ask(1, 1, 2, 2)` повертає 2.
- `ask(2, 4, 4, 4)` повертає 1.
- `ask(3, 1, 3, 2)` повертає 0.

Потім `solve` повертає $\{1, 3, 2, 4\}$.

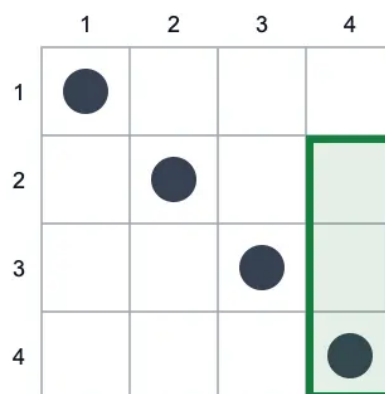
Вихідні дані прикладу грейдера:

1

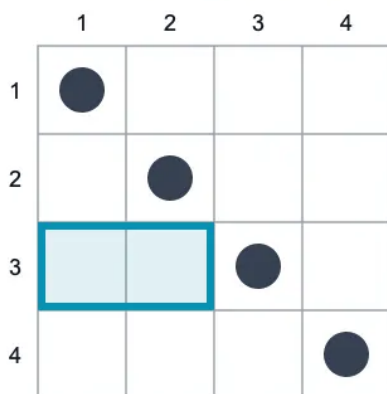
Перший виклик сканує верхній лівий блок 2×2 і знаходить валуни в $(1,1)$ та $(2,2)$. Другий сканує рядки з 2 по 4 стовпця 4 і знаходить валун у $(4,4)$. Третій сканує перші дві клітинки рядка 3 і не знаходить валунів. Повернена область охоплює рядки з 1 по 2 та стовпці з 3 по 4, що є порожньою підсіткою 2×2 .



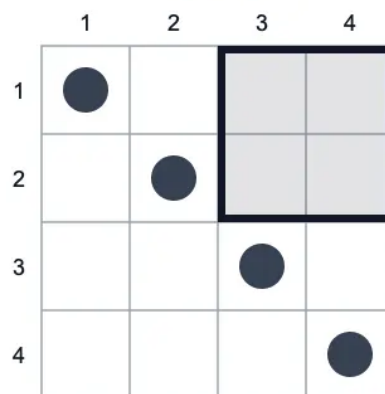
ask(1, 1, 2, 2) returns 2



ask(2, 4, 4, 4) returns 1



ask(3, 1, 3, 2) returns 0



return {1, 3, 2, 4}

Для $N = 4$ програма використала $Q = 3$ викликів, тож $f(Q) = 1.4$, і множник дорівнює 1. Інші відповіді також приймаються, наприклад $\{3, 1, 4, 2\}$, $\{1, 2, 1, 2\}$, $\{3, 2, 3, 2\}$, $\{3, 4, 3, 4\}$ або будь-яка порожня клітинка, оскільки $M = 1$ для $N = 4$.